

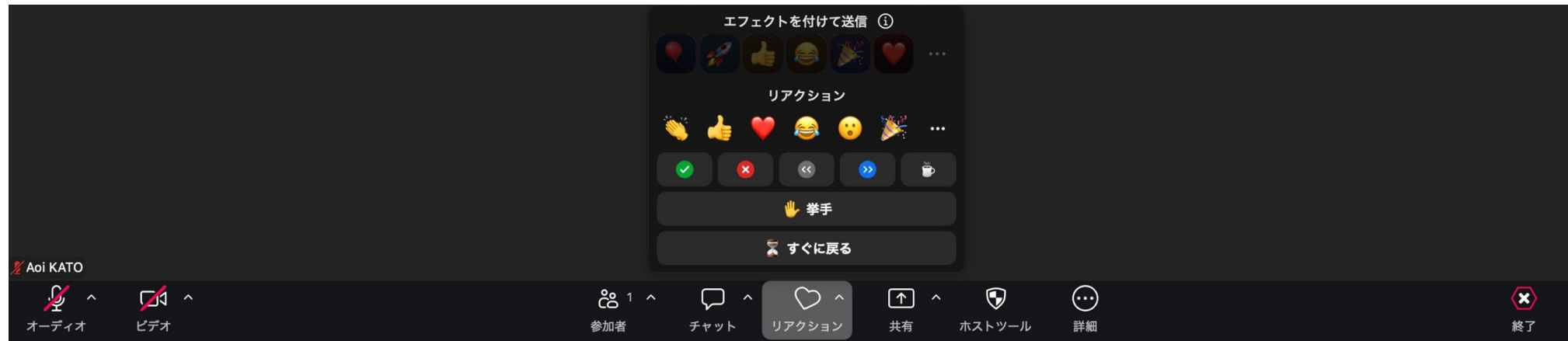
ROOT講習会 第4回 TTreeとデータ構造

横浜国立大学 M2 加藤あおい

2026.07.14



- 前回同様、**初学者向けの想定**です
 - 物足りない方は資料等を見てどんどん前に進んでもらって大丈夫です
 - まずは概念を理解することと・実際自分の解析でどう使うかを想定できることを大事にしてください（調べれば書ける状態）
 - できる方は課題にも挑戦してみてください
- いつでも質問を受け付けます、挙手やリアクションをお願いします



- 受講者Discordで受講後も質問いただければ気づき次第返信させていただきます！



名前：加藤 あおい

所属：横浜国立大学ニュートリノ研 M2

研究内容: Hyper-Kamiokandeの外水槽
検出器/muon fitter

趣味:音楽・お菓子作り(料理)・辛いもの

気軽になんでも聞いてください！



第1回

- コマンドライン環境の基礎
- ROOTとは・そのインストールについて

第2回

- C++の基礎
 - データの型(int(整数値) /double (実数) /char (文字) /ポインタ
 - 制御構文 (if/for/while/break)
- 一次元ヒストグラム (TH1D)

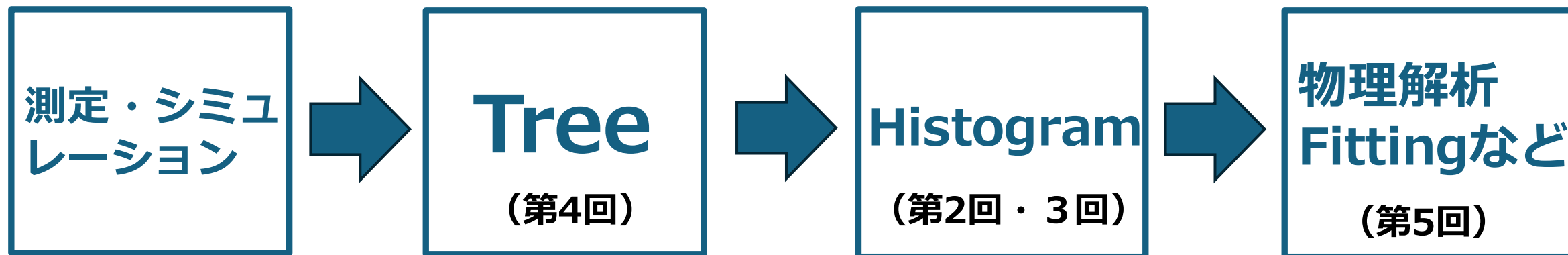
第3回

- 二次元ヒストグラム(TH2D)
- ヒストグラムをROOTファイルに保存する方法 について学んだ

今回の内容

- ヒストグラム (TH1D, TH2D) は作れるようになったけど、
→ 研究においては、測定やシミュレーションで**得られたデータ**を ROOTファイルとして保存し、それを解析する
→ 膨大なデータを扱う際に、**素早くデータを保存する方法が必要**
→ TTree を使う！

実際の研究イメージ



前回まででできるようになったこと

- 乱数（テキスト）をfill して（1D/2D）ヒストグラムにすること
- それを保存すること

今回できるようになって欲しいこと

1. ROOTファイル/TTreeの構造を理解すること
2. 測定データをTreeに保存できるようになること
3. Treeからデータを読んで、ヒストグラムを作れるようになること



1. ROOTファイル/TTree の構造を理解する

ROOTで作成・保存するデータファイルをrootファイルという

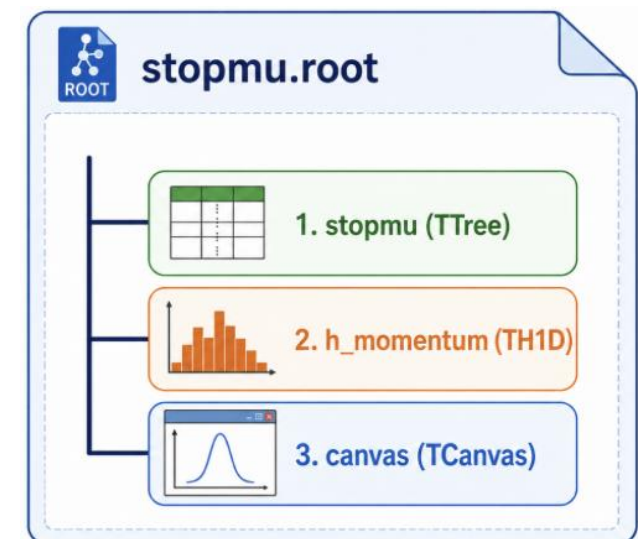
- Wordなら.docx、Excelなら.xlsxのように、ROOTでは.rootで保存する

その中身は

- フォルダのような階層構造になっており、様々なオブジェクトを保存できる
- Ttree・TH1・Tgraph・Tcanvasなどを保存できる
- バイナリ形式なので、テキストより大きなデータを素早く扱える

例とイメージ

```
root -l stopmu.root
root [0] .ls
TFile**   stopmu.root
KEY: TTree   stopmu;1   stop muon vector
KEY: TH1D    h_momentum;1 Momentum
```



データを表形式で保存するためのクラス

- 膨大なイベントデータを効率よく保存することができる

Ttreeでは、データは表形式で格納される

- ExcelやCSVのようなイメージ
- それぞれの列では、データの方が決まっている
- 実際のプログラム中では、intならI,doubleならDなどと表記する

Momentum (float)	dir_x (float)	dir_y (float)	dir_z (float)	pos_x (float)	pos_y (float)	pos_z (float)
436.697449	0.597710	-0.136105	-0.790075	1229.639160	722.505798	1804.699951
635.548035	-0.173315	0.152448	-0.972996	1286.589111	-408.769836	1804.699951
...	...	...	...	...	...	...

- Entry: 一行 (1イベント)
- Branch: 一列 (物理量)
- Leaf: 列の中の値と型 (momentum / Fなど)

	Momentum (float)	dir_x (float)	dir_y (float)	dir_z (float)	pos_x (float)	pos_y (float)	pos_z (float)
entry1 →	436.697449	0.597710	-0.136105	-0.790075	1229.639160	722.505798	1804.699951
entry2 →	635.548035	-0.173315	0.152448	-0.972996	1286.589111	-408.769836	1804.699951
	...	...	...	...	...	...	...
	↑ Branch1	↑ Branch2	↑ Branch3	↑ Branch4	↑ Branch5	↑ Branch6	↑ Branch7

2.測定データをTreeに 保存する



前回までのおさらい : CLIでrootを起動してみる

- ターミナルを起動し、rootと打ち込んでみる

```
Last login: Fri Jul 10 15:49:57 on ttys005
[katouaoi@katouaoiMacBook-Pro ~ % root

-----
| Welcome to ROOT 6.36.02                                     https://root.cern |
| (c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for macosxarm64 on Jul 08 2025, 21:22:35              |
| From tags/6-36-02@6-36-02                                   |
| With Apple clang version 16.0.0 (clang-1600.0.26.6)        |
| Try '.help'/'.'?', '.demo', '.license', '.credits', '.quit'/'.'q' |
-----

root [0] █
```

- 無事起動できている

今回保存するデータ

- 例としてストップミュオンベクターのテキストファイルを用いる (.dat)
- 中身を見てみると...

```
[katouaai@katouaoinoMacBook-Pro rootseminar % cat stopmu.dat
436.697449    0.597710    -0.136105    -0.790075    1229.639160    722.505798    1804.699951
635.548035    -0.173315    0.152448    -0.972996    1286.589111    -408.769836    1804.699951
527.457520    -0.337517    -0.292978    -0.894565    835.746094    -908.049622    1804.699951
403.895752    -0.464044    0.047900    -0.884516    -101.143921    266.851837    1804.699951
522.655823    -0.098860    -0.403991    -0.909405    -892.716003    -537.784973    1804.699951
311.469910    -0.141891    0.366624    -0.919486    386.730286    910.941711    1804.699951
606.693420    -0.120906    -0.153617    -0.980706    421.515747    -1113.536133    1804.699951
257.424133    -0.451026    -0.152544    -0.879378    813.346741    83.635063    1804.699951
373.213135    -0.071657    0.232664    -0.969914    1370.377686    108.534927    1804.699951
699.362732    -0.172879    -0.319177    -0.931794    -435.048035    255.942963    1804.699951
438.203064    -0.105780    -0.478702    -0.871582    970.548218    277.755219    1804.699951
```

1行 = 1イベント、2000行、空白区切りの7列になっている

シミュレーションで作ったdatファイルなので実データではありえないものも入っていますが、今回はあまり気にしないでください

Ttreeを作り、テキストファイルからデータを読み込む

単にテキストを読み込むだけなら TTree::ReadFile が一番早い

make_stopmu_tree_readfile.C の中身

```
#include <iostream>
#include "TFile.h"
#include "TTree.h"

void make_stopmu_tree_readfile(const char* input = "stopmu.dat",
                               const char* output = "stopmu_readfile.root")
{
    TFile* file = new TFile(output, "RECREATE");
    if (!file || file->IsZombie()) {
        std::cerr << "ERROR: cannot create " << output << std::endl;
        return;
    }

    TTree* tree = new TTree("stopmu", "stop muon vector");

    // datの7列を、そのままbranchにする
    // /F は Float_t 型という意味
    Long64_t nread = tree->ReadFile(
        input,
        "momentum/F:dir_x/F:dir_y/F:dir_z/F:pos_x/F:pos_y/F:pos_z/F"
    );

    tree->Write();
    file->Close();

    std::cout << "Created: " << output << std::endl;
    std::cout << "Tree name: stopmu" << std::endl;
    std::cout << "Entries   : " << nread << std::endl;
}
```

向いている場合

- ・列をそのまま保存したい
- ・計算やカットを入れない

make_stopmu_tree_readfile.C の中身

```
// make_stopmu_tree_readfile.C
TFile *file = new TFile("stopmu_readfile.root", "RECREATE");
TTree *tree = new TTree("stopmu", "stop muon vector");

tree->ReadFile("stopmu.dat",
              "momentum/F:dir_x/F:dir_y/F:dir_z/F:pos_x/F:pos_y/F:pos_z/F");

tree->Write();
file->Close();
```

branch名と型を「momentum/F:dir_x/F:...」のように指定する

実際に測定データをTreeに保存してみよう

15/28

実際に実行してみよう

```
[katouaioi@katouaioinoMacBook-Pro rootseminar % root -l -q make_stopmu_tree_readfile.C
```

```
Processing make_stopmu_tree_readfile.C...
Created: stopmu_readfile.root
Tree name: stopmu
Entries : 2000
```

実行コマンド

```
root -l -q
make_stopmu_tree_readfile.C
```

作った中身を見てみよう

```
katouaioi@katouaioinoMacBook-Pro rootseminar % root -l stopmu_readfile.root
root [0]
Attaching file stopmu_readfile.root as _file0...
(TFile *) 0x828a32d80
root [1] stopmu->Print()
*****
*Tree :stopmu : stop muon vector *
*Entries : 2000 : Total = 60323 bytes File Size = 44763 *
* : : Tree compression factor = 1.28 *
*****
*Br 0 :momentum : momentum/F *
*Entries : 2000 : Total Size= 8574 bytes File Size = 7181 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.12 *
*.....*
*Br 1 :dir_x : dir_x/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7463 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 2 :dir_y : dir_y/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7479 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 3 :dir_z : dir_z/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 6777 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.19 *
*.....*
*Br 4 :pos_x : pos_x/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7479 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 5 :pos_y : pos_y/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7488 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 6 :pos_z : pos_z/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 192 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 42.05 *
*.....*
```

```
root [2] stopmu->Scan("momentum:dir_x:dir_y:dir_z:pos_x:pos_y:pos_z")
*****
* Row * momentum * dir_x * dir_y * dir_z * pos_x * pos_y * pos_z *
*****
* 0 * 436.69744 * 0.5977100 * -0.136105 * -0.790075 * 1229.6391 * 722.50579 * 1804.6999 *
* 1 * 635.54803 * -0.173315 * 0.1524479 * -0.972995 * 1286.5891 * -408.7698 * 1804.6999 *
* 2 * 527.45752 * -0.337516 * -0.292977 * -0.894564 * 835.74609 * -908.0496 * 1804.6999 *
* 3 * 403.89575 * -0.464044 * 0.0478999 * -0.884516 * -101.1439 * 266.85183 * 1804.6999 *
* 4 * 522.65582 * -0.098860 * -0.403991 * -0.909404 * -892.7160 * -537.7849 * 1804.6999 *
* 5 * 311.46991 * -0.141891 * 0.3666239 * -0.919485 * 386.73028 * 910.94171 * 1804.6999 *
* 6 * 606.69342 * -0.120906 * -0.153616 * -0.980705 * 421.51574 * -1113.536 * 1804.6999 *
* 7 * 257.42413 * -0.451025 * -0.152544 * -0.879378 * 813.34674 * 83.635063 * 1804.6999 *
* 8 * 373.21313 * -0.071657 * 0.2326640 * -0.969914 * 1370.3776 * 108.53492 * 1804.6999 *
* 9 * 699.36273 * -0.172878 * -0.319177 * -0.931793 * -435.0480 * 255.94296 * 1804.6999 *
* 10 * 438.20306 * -0.105779 * -0.478702 * -0.871581 * 970.54821 * 277.75521 * 1804.6999 *
* 11 * 487.26208 * -0.165972 * -0.193592 * -0.966940 * -58.02495 * -141.7678 * 1804.6999 *
* 12 * 217.01228 * -0.292225 * 0.2655040 * -0.918756 * -973.6450 * 432.96878 * 1804.6999 *
* 13 * 726.06597 * -0.003326 * -0.068085 * -0.997673 * 33.900775 * -530.5609 * 1804.6999 *
* 14 * 520.52795 * -0.028409 * -0.399524 * -0.916281 * 762.47308 *
* 15 * 614.98236 * -0.074015 * -0.581537 * -0.810145 * -693.7581 *
* 16 * 375.08178 * -0.419286 * 0.3360239 * -0.843379 * 1523.5360 *
* 17 * 267.05255 * 0.1474580 * -0.469379 * -0.870598 * 1361.2845 *
* 18 * 484.44714 * -0.021015 * 0.1139409 * -0.993264 * -212.1308 *
* 19 * 581.53125 * 0.1157519 * 0.485073 * -0.866779 * 186.53915 *
* 20 * 569.94201 * 0.0766939 * 0.4557259 * -0.886810 * -674.7443 *
* 21 * 275.57513 * -0.105457 * -0.269181 * -0.957298 * 461.94714 *
* 22 * 490.81384 * -0.423871 * 0.2005340 * -0.883243 * 357.04553 *
* 23 * 390.76647 * 0.1751780 * 0.4614470 * -0.869701 * 1017.2415 *
* 24 * 669.01556 * -0.101159 * 0.1937029 * -0.975830 * 134.68415 *
```

Scan() : TTreeの中身をCLIに表として表示

Print() : branch名・型・entry数を確認

きちんと処理をしてTTreeを作る

計算・条件分岐・単位変換を入れたい場合、ifstream で1行ずつ読み、変数に入れてから Fill する

処理の流れ

1. TFileを開く
2. TTreeを作る
3. Branchを設定する
4. datを1行ずつ読む
5. 必要な計算をする
6. tree->Fill()
7. tree->Write()して閉じる

- ・ 今回のマクロでは元の7列に加えて、entry_id, momentum_GeV (momentum / 1000), r_entry($\sqrt{x^2+y^2+z^2}$) も保存する
- 自分でブランチを作り、解析しやすくすることができる

実際に実行してみよう

```
katouaioi@katouaioinoMacBook-Pro rootseminar % root -l -q make_stopmu_tree.C
Processing make_stopmu_tree.C...
Created: stopmu.root
Tree name: stopmu
Entries : 2000
katouaioi@katouaioinoMacBook-Pro rootseminar %
```

実行コマンド

root -l -q make_stopmu_tree.C

出力：stopmu.root / TTree名：stopmu

実際に測定データをTreeに保存してみよう

17/28

1.Tfileを開く

```
TFFile* file = new TFFile(output, "RECREATE");  
if (!file || file->IsZombie()) {  
    std::cerr << "ERROR: cannot create " << output << std::endl;  
    return;  
}
```

2.Ttreeを作る

```
TTTree* tree = new TTTree("stopmu", "stop muon vector");
```

3.Tbranchを設定する

TTTreeに列 (branch) を作り、値を格納する変数のアドレスを渡す

```
// datから読む7列  
Float_t momentum = 0; // [MeV/c]  
Float_t dir_x = 0;  
Float_t dir_y = 0;  
Float_t dir_z = 0;  
Float_t pos_x = 0; // [cm]  
Float_t pos_y = 0; // [cm]  
Float_t pos_z = 0; // [cm]
```

```
// 自分で計算して追加するbranch  
Int_t entry_id = 0;  
Float_t momentum_GeV = 0; // [GeV/c]  
Float_t r_entry = 0; // [cm]
```

```
tree->Branch("entry_id", &entry_id, "entry_id/I");  
tree->Branch("momentum", &momentum, "momentum/F");  
tree->Branch("dir_x", &dir_x, "dir_x/F");  
tree->Branch("dir_y", &dir_y, "dir_y/F");  
tree->Branch("dir_z", &dir_z, "dir_z/F");  
tree->Branch("pos_x", &pos_x, "pos_x/F");  
tree->Branch("pos_y", &pos_y, "pos_y/F");  
tree->Branch("pos_z", &pos_z, "pos_z/F");  
tree->Branch("momentum_GeV", &momentum_GeV, "momentum_GeV/F");  
tree->Branch("r_entry", &r_entry, "r_entry/F");
```

Branchの第1引数：保存
する名前
第2引数：変数のアドレス
第3引数：branch名/型

型の例：
/F = Float_t
/D = Double_t
/I = Int_t


```
while (fin >> momentum >> dir_x >> dir_y >> dir_z >> pos_x >> pos_y >> pos_z) {  
    // 例：方向ベクトルを念のため規格化する  
    const Float_t norm = std::sqrt(dir_x*dir_x + dir_y*dir_y + dir_z*dir_z);  
    if (norm > 0) {  
        dir_x /= norm;  
        dir_y /= norm;  
        dir_z /= norm;  
    }  
  
    // 例：解析しやすい量を追加で計算して保存する  
    momentum_GeV = momentum / 1000.0;  
    r_entry = std::sqrt(pos_x*pos_x + pos_y*pos_y + pos_z*pos_z);  
  
    tree->Fill();  
    entry_id++;  
}  
  
tree->Write();  
file->Close();
```

4.datを一行ずつ読む

5.必要な計算をする

6. tree->Fill()

繰り返す

その時点の変数の値が1行として
保存される

7. tree->Write()して
閉じる

実際に測定データをTreeに保存してみよう

19/28

作った中身を見てみよう

```
Entries : 2000
[katouaioi@katouaioinoMacBook-Pro rootseminar % root -l stopmu.root
root [0]
Attaching file stopmu.root as _file0...
(TFile *) 0xc19964e00
[root [1] stopmu->Print()]
*****
*Tree :stopmu : stop muon vector *
*Entries : 2000 : Total Size= 86082 bytes File Size = 62087 *
* : : Tree compression factor = 1.32 *
*****
*Br 0 :entry_id : entry_id/I *
*Entries : 2000 : Total Size= 8574 bytes File Size = 2910 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 2.78 *
*.....*
*Br 1 :momentum : momentum/F *
*Entries : 2000 : Total Size= 8574 bytes File Size = 7181 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.12 *
*.....*
*Br 2 :dir_x : dir_x/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7474 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 3 :dir_y : dir_y/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7484 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 4 :dir_z : dir_z/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 6797 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.19 *
*.....*
*Br 5 :pos_x : pos_x/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7479 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 6 :pos_y : pos_y/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 7488 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.08 *
*.....*
*Br 7 :pos_z : pos_z/F *
*Entries : 2000 : Total Size= 8559 bytes File Size = 192 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 42.05 *
*.....*
*Br 8 :momentum_GeV : momentum_GeV/F *
*Entries : 2000 : Total Size= 8594 bytes File Size = 7171 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.13 *
*.....*
*Br 9 :r_entry : r_entry/F *
*Entries : 2000 : Total Size= 8569 bytes File Size = 6986 *
*Baskets : 1 : Basket Size= 32000 bytes Compression= 1.16 *
*.....*
```

```
.....
[root [2] stopmu->Scan("entry_id:momentum:momentum_GeV:r_entry", "", "", 10)
*****
* Row * entry_id * momentum * momentum_ * r_entry *
*****
* 0 * 0 * 436.69744 * 0.4366974 * 2300.2106 *
* 1 * 1 * 635.54803 * 0.6355480 * 2253.7404 *
* 2 * 2 * 527.45752 * 0.5274575 * 2186.3137 *
* 3 * 3 * 403.89575 * 0.4038957 * 1827.1240 *
* 4 * 4 * 522.65582 * 0.5226558 * 2084.0097 *
* 5 * 5 * 311.46991 * 0.3114699 * 2058.2314 *
* 6 * 6 * 606.69342 * 0.6066934 * 2162.0776 *
* 7 * 7 * 257.42413 * 0.2574241 * 1981.2797 *
* 8 * 8 * 373.21313 * 0.3732131 * 2268.6245 *
* 9 * 9 * 699.36273 * 0.6993627 * 1873.9571 *
*****
(long long) 10
```

Scan() : TTreeの中身をCLIに表として表示

Print() : branch名・型・entry数を確認



3. Treeからデータを読み出し、 ヒストグラムを作る

読み出して表示する流れ

1. ROOTファイルを開く
2. TTreeをGetObjectで取得
3. SetBranchAddressでbranchと変数を接続
4. GetEntry(i) を呼ぶと、i番目のentryの値がSetBranchAddress した変数に入る

```
TFile* file = TFile::Open(rootfile, "READ");
if (!file || file->IsZombie()) {
    std::cerr << "ERROR: cannot open " << rootfile << std::endl;
    return;
}

TTree* tree = nullptr;
file->GetObject("stopmu", tree);
if (!tree) {
    std::cerr << "ERROR: TTree 'stopmu' was not found" << std::endl;
    return;
}

tree->Print();
tree->Scan("entry_id:momentum:dir_x:dir_y:dir_z:pos_x:pos_y:pos_z:momentum_GeV:r_entry", "", "", nshow);
```

```
Int_t entry_id = 0;
Float_t momentum = 0, dir_x = 0, dir_y = 0, dir_z = 0;
Float_t pos_x = 0, pos_y = 0, pos_z = 0;
Float_t momentum_GeV = 0, r_entry = 0;

tree->SetBranchAddress("entry_id", &entry_id);
tree->SetBranchAddress("momentum", &momentum);
tree->SetBranchAddress("dir_x", &dir_x);
tree->SetBranchAddress("dir_y", &dir_y);
tree->SetBranchAddress("dir_z", &dir_z);
tree->SetBranchAddress("pos_x", &pos_x);
tree->SetBranchAddress("pos_y", &pos_y);
tree->SetBranchAddress("pos_z", &pos_z);
tree->SetBranchAddress("momentum_GeV", &momentum_GeV);
tree->SetBranchAddress("r_entry", &r_entry);

const Long64_t nentries = tree->GetEntries();
const Long64_t nloop = std::min(nentries, nshow);

std::cout << "\nFirst " << nloop << " entries by GetEntry():\n";
for (Long64_t i = 0; i < nloop; i++) {
    tree->GetEntry(i);
    std::cout << "entry=" << entry_id
                << " momentum=" << momentum << " MeV/c"
                << " dir=(" << dir_x << ", " << dir_y << ", " << dir_z << ")"
                << " pos=(" << pos_x << ", " << pos_y << ", " << pos_z << ") cm"
                << " r=" << r_entry << " cm"
                << std::endl;
}

file->Close();
```


実際にTreeを読み出してみよう

22/28

実際に実行してみよう

```
katouaui@katouauiMacBook-Pro rootseminar % root -l -q scan_stopmu_tree.C

Processing scan_stopmu_tree.C...
*****
*Tree      :stopmu      : stop muon vector *
*Entries   :    2000    : Total =          86082 bytes File Size =        62087 *
*          :          : Tree compression factor =    1.32 *
*****
*Br    0 :entry_id : entry_id/I *
*Entries :    2000 : Total Size=         8574 bytes File Size =         2910 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    2.78 *
*.....*
*Br    1 :momentum : momentum/F *
*Entries :    2000 : Total Size=         8574 bytes File Size =         7181 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.12 *
*.....*
*Br    2 :dir_x    : dir_x/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =         7474 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.08 *
*.....*
*Br    3 :dir_y    : dir_y/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =         7484 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.08 *
*.....*
*Br    4 :dir_z    : dir_z/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =         6797 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.19 *
*.....*
*Br    5 :pos_x    : pos_x/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =         7479 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.08 *
*.....*
*Br    6 :pos_y    : pos_y/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =         7488 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.08 *
*.....*
*Br    7 :pos_z    : pos_z/F *
*Entries :    2000 : Total Size=         8559 bytes File Size =          192 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=   42.05 *
*.....*
*Br    8 :momentum_GeV : momentum_GeV/F *
*Entries :    2000 : Total Size=         8594 bytes File Size =         7171 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.13 *
*.....*
*Br    9 :r_entry   : r_entry/F *
*Entries :    2000 : Total Size=         8569 bytes File Size =         6986 *
*Baskets :        1 : Basket Size=        32000 bytes Compression=    1.16 *
*.....*
```

実行コマンド

```
root -l -q scan_stopmu_tree.C
```

出力 : stopmu.root / TTree名 : stopmu

```
*****
* Row * entry_id * momentum * dir_x * dir_y * dir_z * pos_x * pos_y * pos_z * momentum * r_entry *
*****
* 0 * 0 * 436.69744 * 0.5977099 * -0.136104 * -0.790074 * 1229.6391 * 722.50579 * 1804.6999 * 0.4366974 * 2300.2106 *
* 1 * 1 * 635.54803 * -0.173315 * 0.1524480 * -0.972996 * 1286.5891 * -408.7698 * 1804.6999 * 0.6355480 * 2253.7404 *
* 2 * 2 * 527.45752 * -0.337516 * -0.292977 * -0.894564 * 835.74609 * -908.0496 * 1804.6999 * 0.5274575 * 2186.3137 *
* 3 * 3 * 403.89575 * -0.464044 * 0.0479000 * -0.884516 * -101.1439 * 266.85183 * 1804.6999 * 0.4038957 * 1827.1240 *
* 4 * 4 * 522.65582 * -0.098860 * -0.403991 * -0.909405 * -892.7160 * -537.7849 * 1804.6999 * 0.5226558 * 2084.0097 *
* 5 * 5 * 311.46991 * -0.141890 * 0.3666238 * -0.919485 * 386.73028 * 910.94171 * 1804.6999 * 0.3114699 * 2058.2314 *
* 6 * 6 * 606.69342 * -0.120905 * -0.153616 * -0.980705 * 421.51574 * -1113.536 * 1804.6999 * 0.6066934 * 2162.0776 *
* 7 * 7 * 257.42413 * -0.451026 * -0.152544 * -0.879378 * 813.34674 * 83.635063 * 1804.6999 * 0.2574241 * 1981.2797 *
* 8 * 8 * 373.21313 * -0.071656 * 0.2326639 * -0.969913 * 1370.3776 * 108.53492 * 1804.6999 * 0.3732131 * 2268.6245 *
* 9 * 9 * 699.36273 * -0.172878 * -0.319176 * -0.931793 * -435.0480 * 255.94296 * 1804.6999 * 0.6993627 * 1873.9571 *
*****

First 10 entries by GetEntry():
entry=0 momentum=436.697 MeV/c dir=(0.59771, -0.136105, -0.790075) pos=(1229.64, 722.506, 1804.7) cm r=2300.21 cm
entry=1 momentum=635.548 MeV/c dir=(-0.173315, 0.152448, -0.972996) pos=(1286.59, -408.77, 1804.7) cm r=2253.74 cm
entry=2 momentum=527.458 MeV/c dir=(-0.337517, -0.292978, -0.894565) pos=(835.746, -908.05, 1804.7) cm r=2186.31 cm
entry=3 momentum=403.896 MeV/c dir=(-0.464044, 0.0479, -0.884516) pos=(-101.144, 266.852, 1804.7) cm r=1827.12 cm
entry=4 momentum=522.656 MeV/c dir=(-0.09886, -0.403991, -0.909405) pos=(-892.716, -537.785, 1804.7) cm r=2084.01 cm
entry=5 momentum=311.47 MeV/c dir=(-0.141891, 0.366624, -0.919486) pos=(386.73, 910.942, 1804.7) cm r=2058.23 cm
entry=6 momentum=606.693 MeV/c dir=(-0.120906, -0.153617, -0.980706) pos=(421.516, -1113.54, 1804.7) cm r=2162.08 cm
entry=7 momentum=257.424 MeV/c dir=(-0.451026, -0.152544, -0.879378) pos=(813.347, 83.6351, 1804.7) cm r=1981.28 cm
entry=8 momentum=373.213 MeV/c dir=(-0.071657, 0.232664, -0.969914) pos=(1370.38, 108.535, 1804.7) cm r=2268.62 cm
entry=9 momentum=699.363 MeV/c dir=(-0.172879, -0.319177, -0.931793) pos=(-435.048, 255.943, 1804.7) cm r=1873.96 cm
```

Tree → drawでその列のヒストグラムを生成できる

```
TTree* tree = nullptr;
file->GetObject("stopmu", tree);
if (!tree) {
    std::cerr << "ERROR: TTree 'stopmu' was not found" << std::endl;
    return;
}

TCanvas* c = new TCanvas("c_momentum", "momentum", 800, 600);
TH1D* h = new TH1D("h_momentum",
    "Muon momentum;Momentum [MeV/c];Entries",
    50, 0, 800);

tree->Draw("momentum>>h_momentum");
h->Draw();

c->SaveAs("momentum_hist.png");
```

momentum branchを使って、運動量分布を描く

Drawの基本形

tree->Draw("branch名>>hist
名")

ヒストグラムのbin幅や範囲はTH1Dで決める

実行してみよう

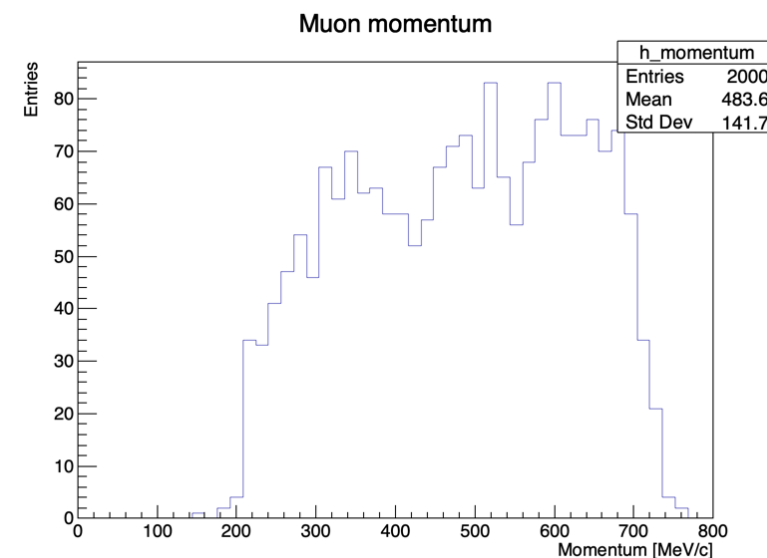
実行コマンド

```
root -l -q draw_momentum_hist_cut.C
```

出力：momentum_hist_cut.root / TTree名：momentum_hist_cut.png

```
katouaoi@katouaoiMacBook-Pro rootseminar % root -l -q draw_momentum_hist.C
```

```
Processing draw_momentum_hist.C...
Info in <TCanvas::Print>: file momentum_hist.png has been created
Saved: momentum_hist.png, momentum_hist.root
```



カットをかける

```
// cut例：かなり下向きで、上側から入ってくるイベント  
TCut cut = "dir_z < -0.8 && pos_z > 0";  
tree->Draw("momentum>>h_momentum_cut", cut);  
h->Draw();
```

実行してみよう

実行コマンド

```
root -l -q draw_momentum_hist_cut.C
```

出力：momentum_hist_cut.root / TTree名：momentum_hist_cut.png

```
[katouaioi@katouaioinoMacBook-Pro rootseminar % root -l -q draw_momentum_hist_cut.C
```

```
Processing draw_momentum_hist_cut.C...  
Info in <TCanvas::Print>: file momentum_hist_cut.png has been created  
Cut : dir_z < -0.8 && pos_z > 0  
Saved: momentum_hist_cut.png, momentum_hist_cut.root
```

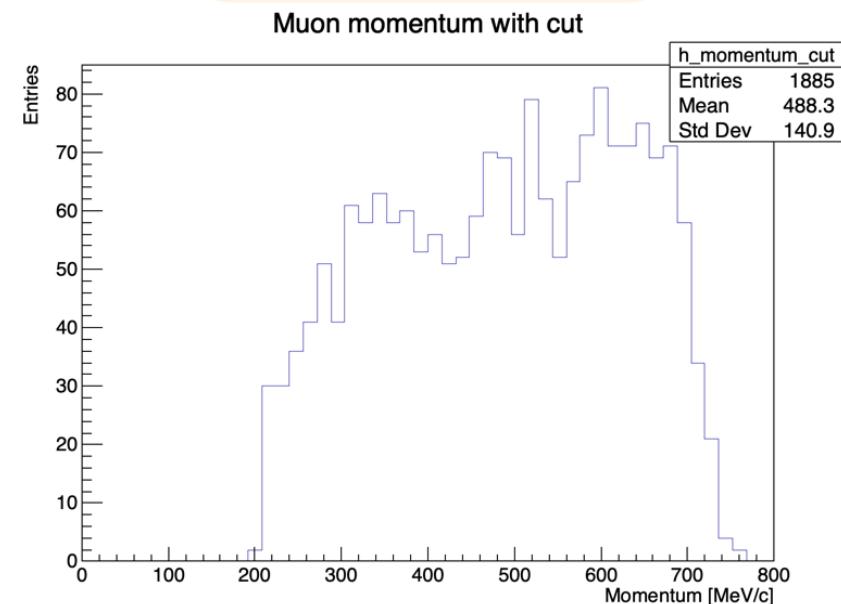
今回のcut例

$\text{dir_z} < -0.8$

→かなり下向きのイベント

$\text{pos_z} > 0$

→上側から入ってくるイベント





4.演習問題

- ①stopmu.rootのmomentum branch のみを
書き出して、stopmu_mom.root として保存してみよう
- ②pos_x,pos_yの2Dヒストグラムを書いてみよう

より簡単に解析するためのツール

- **Pyroot**

ROOTのクラスをPythonから使うための仕組み

TFile, TTree, TH1DなどをPythonで扱える

NumPy・matplotlibなどと組み合わせやすい

例：file = ROOT.TFile("stopmu.root") など表記が変わる

- **RDataFrame**

TTreeを「表データ」のように扱う解析用クラス

Filterでカット、Defineで新しい列を作れる

Histo1Dでヒストグラムを簡単に作成できる

自分でevent loopを書かずに解析できる

ぜひ調べて使ってみてください！

おわりに

Ttree・データの構造についての知識を深めることができたでしょうか？

これ以降もぜひdiscordなどで質問してください！

YMAPの会員登録・運営ご協力も興味のある方はぜひお願いします

